AI in the Loop: Hardware Design When Intelligence Is Cheap

AI is getting very good at anything that can be checked inside a computer, and much of engineering can be. What can't be checked that way has to be built and tested. This is the workflow I use to get an agent's speed on the first kind of work without losing track of which numbers to trust, and the argument behind it.

MIT · FALL 2026

SAMUEL KIRSCHNER · SAMKIR.COM/PROJECTS/AI-IN-THE-LOOP/

01

The argument

Dwarkesh Patel published an essay this week arguing that AI improves explosively wherever answers can be checked and a model can practice thousands of times. Math is his example. In 2023 a frontier model couldn't reliably add four-digit numbers. In September, OpenAI reported that an internal model, running about 10,000 agents for 88 hours, had resolved the Navier–Stokes Millennium Prize problem, with a proof formalized in Lean so that a computer can check every step. Mathematicians are still reviewing it, but the slope is hard to argue with.

Much of engineering can be checked the same way. A CAD kernel can tell whether constraints are satisfied and whether parts interfere, and an analysis can be checked against units, conservation laws, limiting cases and an independent finite-element run. The usual objections, that AI can't reason spatially, makes up numbers and lacks an engineer's judgment, sound a lot like what people said about math a few years ago. I expect the computational half of engineering, meaning derivations, models, CAD, simulation and code, to go the way of math, and soon.

The other half is checked in the physical world. There's a small irony in the Navier–Stokes result here: the proof shows the equations allow a fluid's speed to become infinite in finite time, which no real fluid does. It settles a question about the model. Whether the valve in my reactor leaks is still a question about the world, and answering it means building something and testing it. So as AI gets cheaper and better, I think the bottleneck in engineering moves to the boundary between models and the world, and the rest of this piece is about the workflow I've built around that boundary.

02

Discovery and integration

Patel expects that an AI with remote access to a materials lab would be better than humanity at finding a room-temperature superconductor. Suppose it is. The discovery wouldn't produce a single motor, MRI scanner or fusion magnet. Someone would still have to learn how the material fails, how to make it into tape by the kilometer, how to join and cool it, and how to redesign each machine around it.

History suggests how long that takes. YBCO, the first superconductor to work above the boiling point of liquid nitrogen, was discovered in 1987. Its tape form, REBCO, became the conductor in the fusion magnet that MIT's Plasma Science and Fusion Center and Commonwealth Fusion Systems ramped to 20 tesla in 2021, 34 years later, after what the team described as a rework of almost every principle of superconducting magnet design.

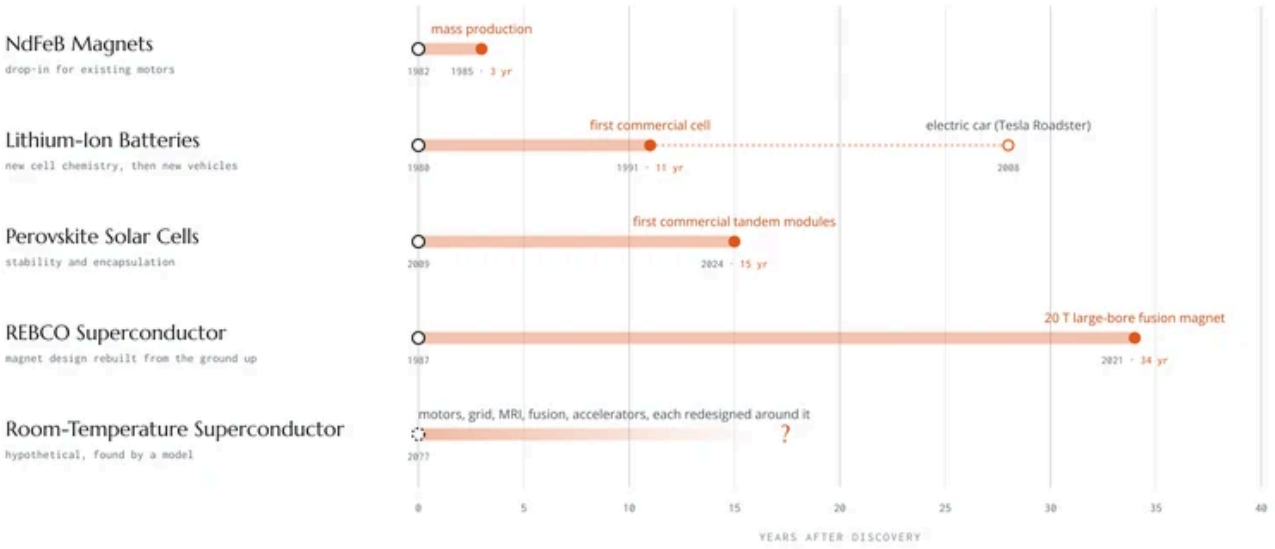


FIG. 01 Years from the discovery of a material to the first engineered system built around it at scale. Milestones are my choice of a representative first system, and other choices would move the bars by a few years. The gaps are all engineering: processing, joining, failure modes, and redesign of the surrounding machine.

The lag scales with how much of the surrounding system has to change. Neodymium-iron-boron magnets reached mass production about three years after their discovery in 1982 because they dropped into motors that already used weaker magnets. If AI speeds up discovery across materials, batteries, actuators and electronics at once, each discovery changes the constraints that thousands of existing designs were built around. I expect AI to increase the number of engineering problems worth solving faster than it solves them.

03

Where AI will be slower

Math and code share one property: the verifier runs in a computer. Most of engineering's verifiers don't.

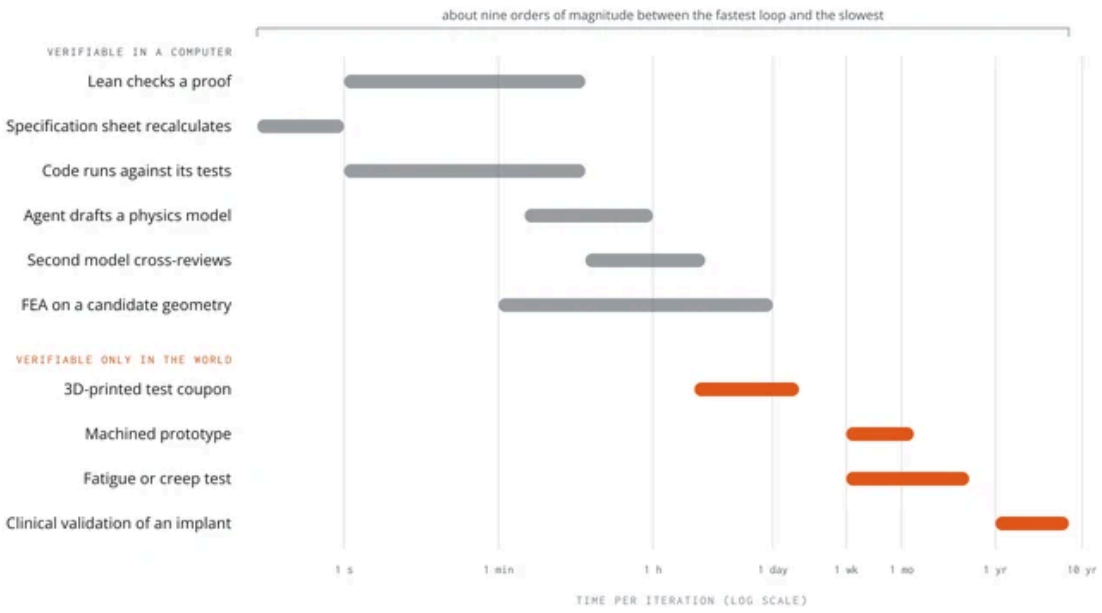


FIG. 02 Rough time per iteration for the feedback loops in hardware development, on a log scale. Ranges are order-of-magnitude estimates from my own work and common practice. The loops in gray can be checked inside a computer, which is where AI is already strong. Every loop that can validate a physical design is orange and sits to the right of them.

The parts that can only be checked in the world will resist automation longest, for a few reasons.

- **Time can't be parallelized.** Fatigue, creep, corrosion and the aging of an implant take months or years, and accelerated tests depend on a model of the very failure being tested.
- **Every test costs material.** A thousand parallel experiments need a thousand rigs, which is capital rather than compute.
- **Much of the physics was never written down.** How a silicone cures in a humid week, how a seal wears in, how tissue reacts to an implant: contact, wear, manufacturing variation and biology are the hardest things to simulate, and most of the data on them sits in shop notes and unpublished failure reports.

- **The verifier has to be built.** Lean already exists for math. In hardware the verifier is a test rig and a calibration, and designing a measurement that can't fool you is engineering work in its own right.
- **Someone has to sign.** Pressure vessels and implants need a person accountable for them.

Materials science has already had a preview of what happens when generation outruns verification: claims of millions of AI-predicted crystals and dozens of robot-synthesized compounds were contested within months by chemists who checked them by hand. The obvious fix is to automate the lab, but a robot that can do a technician's work needs dexterous hands, reliable actuators and compact power, each of which is a hardware problem bound by the same slow loops. Robotics will get there, but it is gated by the processes it is trying to automate.

04

What this means for engineers

If that's right, the computational work of engineering becomes cheap, and the scarce work is deciding what to build, writing requirements sharp enough to rank a hundred candidate designs, building the rigs and error budgets that turn a physical question into a trustworthy number, choosing the tests that eliminate the most candidates per dollar, and signing for the result. Fundamentals matter more in that world, because they are how you notice that a model is missing the physics that matters and design the test that would expose it.

Two cautions. More engineering work doesn't guarantee more engineers: ten times the work at a hundred times the productivity is a tenth of the headcount. And the bottleneck won't stay put; as robotics matures it moves again, toward manufacturing capacity, regulation and capital.

05

A workflow for now

The argument above is a prediction. What I can test today is how I work. The workflow I've settled into covers the early part of design, from requirements to a model I trust enough to build from, and it is organized around one fact: an agent can produce numbers faster than I can check them, so the structure has to make checking cheap and keep the hardware loop in charge.

Most writing about working with AI comes from people who build software, where the loop is short. My test suite is a part that has to be printed, cured, assembled and measured, so a wrong number can hide for days before it shows up as a leak. I want the agent's speed without giving up the thing a mechanical engineer can't do without, which is knowing where every number came from.

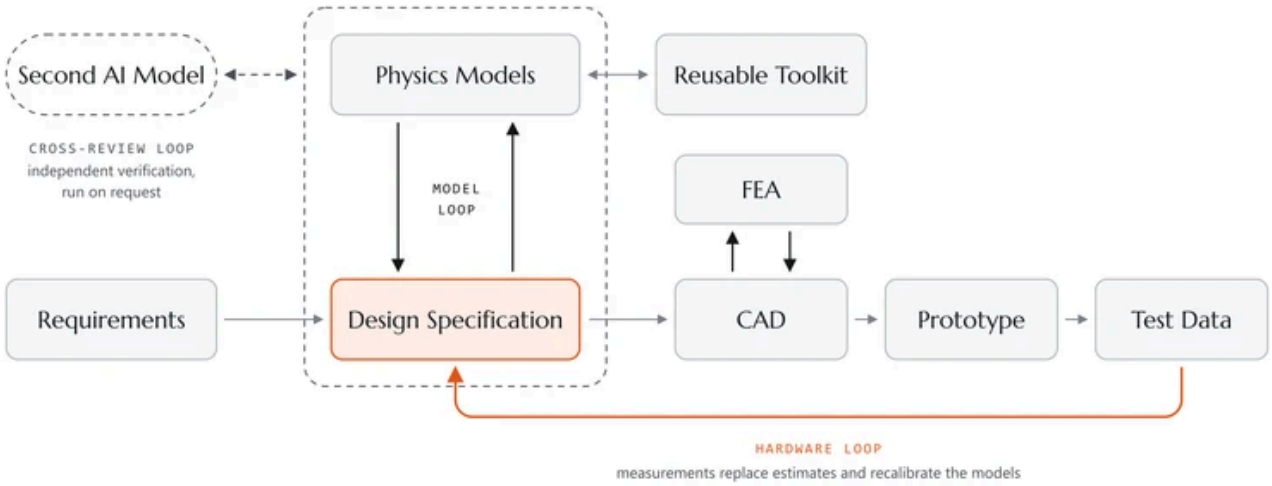


FIG. 03 The workflow as loops around the design specification. The model loop is fast and runs constantly, the cross-review loop runs on request, and the hardware loop is the slowest and the only one that validates anything.

As a running example I'll use a 15 mm aluminum-water reactor I'm developing, a capsule that meters water onto aluminum to make hydrogen for untethered soft

robots. I describe it only in outline because the work isn't published yet.

Requirements

Every project starts as a conversation. Before anything is calculated, I use the AI to argue with me about what the system has to do and what constrains it, starting broad and challenging my assumptions. From there we look for prior art in papers, commercial products and patents, because if someone has already built the wheel I would like to know before I design a worse one. When the requirements really do demand something new, I ask for several parallel concepts before converging on one.

In this stage the AI is a good sparring partner and a poor decision maker. It doesn't know which constraint I would trade away under pressure, and it has never been surprised by a prototype, so the requirements and the choice of architecture stay with me.

06

The specification sheet

The center of the workflow is a spreadsheet I call the design specification. It works as a numerical twin of the design: every geometry, load, material property and assumption lives in it, every derived quantity is a live formula, and the CAD is built from it.

Variable	Value	±	Units	Source	Notes
Packaging					
package OD	15	0.05	mm	15 mm build; 9 mm too hard to fabricate	
wall thickness	1	0.05	mm		outer wall, cap roof, base
package ID (= reservoir ID)	13	0.05	mm	=MROUND(D_pkg-2*t_pkg,res_print)	OD - 2 walls
reactor length	24	0.14	mm	=t_cap+t_vl+t_dl+(R_bl+L_cyl)+t_bl+…	
Reaction (water fixed by the reservoir)					
reaction efficiency	0.9	0.05			fraction of loaded Al that reacts
aluminum mass	0.95	0.05	g	=n_water/(2*lam*eta)*26.9815	Al + 2H2O -> AlOOH + 1.5H2
hydrogen volume	1168	13	mL	=1.5*n_rxn*24465	25 C, 1 atm
Pump Checks					
margin, rest leak	0.37	0.21	kPa	=2*P_crack-0.00978*L_reactor	PASS | no flow at rest, incl. tolerance
margin, up-stroke to lip	0.81	0.11	mm	=h_lip_dia-delta	PASS | hub stays off the cup, incl. tolerance

INPUT
a value I chose, with the reason written beside it

DERIVED
a live formula, in red, that updates when an input moves

fx SOURCE
FORMULATEXT prints each formula next to its result

CHECK
passes only when the margin is larger than its uncertainty

FIG. 04 An excerpt from the reactor's specification sheet, redrawn with values rounded to their uncertainty.

Inputs are black and carry a source, which can be a datasheet, a requirement, or the reason I chose the value. Derived values are red, and beside each one Excel's `FORMULATEXT` function prints the formula that produced it, so the calculation can be read without clicking into a cell. Most values also carry an uncertainty, drawn from a separate sheet that budgets the error from fabrication, assembly, purchased parts and measurement. The sheet ends in checks, each of which passes only when its margin is larger than its own uncertainty. I also keep the number of decimal places honest about how well a value is known, which needs watching because an agent will happily report a stiffness to six digits.

Two habits from precision machine design are built into it. The uncertainty sheet is an error budget, the same tool a machine designer uses to decide which error sources deserve attention, applied here to a whole device. The source column carries the habit of Slocum's FRDPARRC tables, which pair every functional requirement and design parameter with its analysis, references and risks, so that no number enters the design without a reason written next to it.

For the reactor, the sheet holds 58 inputs and 107 derived values across dimensions, forces, pump dynamics and the drive coil, and it ends in a summary page of 14 pass or fail checks. If I change one wall thickness, everything downstream moves and the summary tells me what broke.

An agent could build me an interactive app with sliders in the same time, and those interfaces are popular because they make good demos, but they hide the arithmetic. A spreadsheet shows its work, so when a number looks wrong I can follow it back through every cell that fed it, and so can anyone I hand the file to. That matters more as more of the arithmetic is written by an agent.

07

Agent-built models

With the specification in place, the agent builds the physics. I describe the dominant effects and it writes the model: stoichiometry for the reaction, the bending and stretching of a diaphragm, pressure drops through a valve, the force between a coil and a magnet, and the heat the coil sheds. I could derive each of these by hand from a textbook, slowly, but the agent drafts them quickly enough that I can explore a design space instead of defending the first design that works.

Cheap models also change how I choose an architecture. Hydrogen can power a soft robot in several ways: the gas can expand directly as the working fluid, a fuel cell can convert it to electricity that drives a pump, or the two can be combined. When each model costs an afternoon, I don't have to commit to one of these early and defend it. I can have the agent build all of them against the same constraints from the sheet and see which combinations of physics make sense at a given size and duty cycle.

Excel has been enough for everything in the reactor. When a model outgrows it I move to Python, and there I prefer Jupyter notebooks to loose scripts for the same

reason I prefer spreadsheets to apps: a notebook keeps the equations, assumptions, code and outputs together in the order they were computed.

Some of these calculations are useful beyond one project. Once one of them has been checked, such as a bolted joint, a shaft fit or a motor sizing, it goes into a toolkit and comes along to the next project. Over time the toolkit is where the speed compounds, because each verified tool is one less thing an agent has to derive from scratch and one less thing I have to check again.

08

Cross-review

The output of a single AI model is low confidence, however sure it sounds, so for anything that matters I bring in a second model from a different lab. It first works out how it would attack the problem without looking at the existing solution, and only then reviews that solution critically. The two models keep a shared review log in the project folder, which lets either one pick up where the other left off. Because all of this costs time and money, it runs only when I ask.

An early version of the reactor model was written in Python, and it ran without complaint. When the second model reviewed it, it found six groups of defects and reproduced each one before reporting it. A sensitivity study was returning zeros because it looked up results by display labels that had since been renamed. The water dose was computed and then never passed to the gas generation model, so a 1 ms pulse and a 100 ms pulse gave identical hydrogen curves. The dose formula also jumped by a factor of two at one particular pulse width. When the first model looked at the evidence, it agreed with all three.

None of those errors would have crashed anything, and each would have produced a plausible plot. That is why the working agreement both agents now follow

includes the line that numerical agreement and passing checks are not physical validation.

Metrology has a useful way to think about this. When you measure a spindle's error with a precision test ball, the reading mixes the spindle's error motion with the ball's own out-of-roundness, and repeating the measurement never separates them. Donaldson's reversal, from 1972, measures a second time with the ball and the probe both turned 180 degrees, so one error changes sign and the other doesn't, and the two can be solved for. Cross-review is a loose version of the same idea for models. A model trained differently has different blind spots, so where two models disagree at least one is wrong, and the disagreement shows me where to look. Agreement is weaker evidence than it seems, though, because models from different labs learn from much of the same text and can share the same mistakes. Unlike the spindle and the ball, their errors are not independent.

09

Hardware in the loop

Two models agreeing still isn't the same as being right, which leaves hardware as the outermost loop and the only one that validates anything. It is the verifier from the argument above, and the part of the workflow I expect to matter more over time.

What this loop looks like changes with every project, but in all of them every estimate in the specification is labeled as an estimate, and testing replaces estimates with measurements. In the reactor's error budget, for example, the print tolerance began as a guess, and a small printed coupon of pins, holes and thin walls turns that guess into a number with scatter. Once a measurement goes into the sheet, the uncertainty on everything downstream of it shrinks or grows, and the checks re-evaluate themselves.

The loop earns its keep when a test disagrees with a prediction. The useful response is to find which assumption failed, change it in the sheet where every downstream number can see it, and decide what to measure next. That cycle of prediction, test and correction is the part of engineering I expect to stay expensive the longest, and it's the part this workflow is built to protect.

10

CAD and design intent

AI for CAD is starting to be real. I've found it useful for standard parts such as a bracket or a simple housing, which it can model quickly from a drawing, but for complex parts it isn't there yet. Spatial reasoning is still the weak point, and a model that writes fluent code can lose track of which face a feature sits on.

Part of that weakness, I suspect, is bandwidth. Describing a three-dimensional layout in text is hard for anyone, which is why engineers rarely describe geometry to each other in paragraphs. They sketch it, point at it and mark up drawings. A CAD assembly also holds information that never appears in a prompt: relative positions, mates, degrees of freedom, clearances and interfaces.

Here is the interface I'd want. I sit at a desk with a text window and a drawing tablet. I upload reference drawings for the purchased parts, describe the object in a few sentences, and sketch the rough layout. The agent compiles all of that into a parametric model, which appears on the tablet, and I edit it the way I'd mark up a print. I circle a flange and write "stiffer, no more than 20 g," draw a dimension where one should be fixed, or draw a line between two bores to say they stay coaxial.

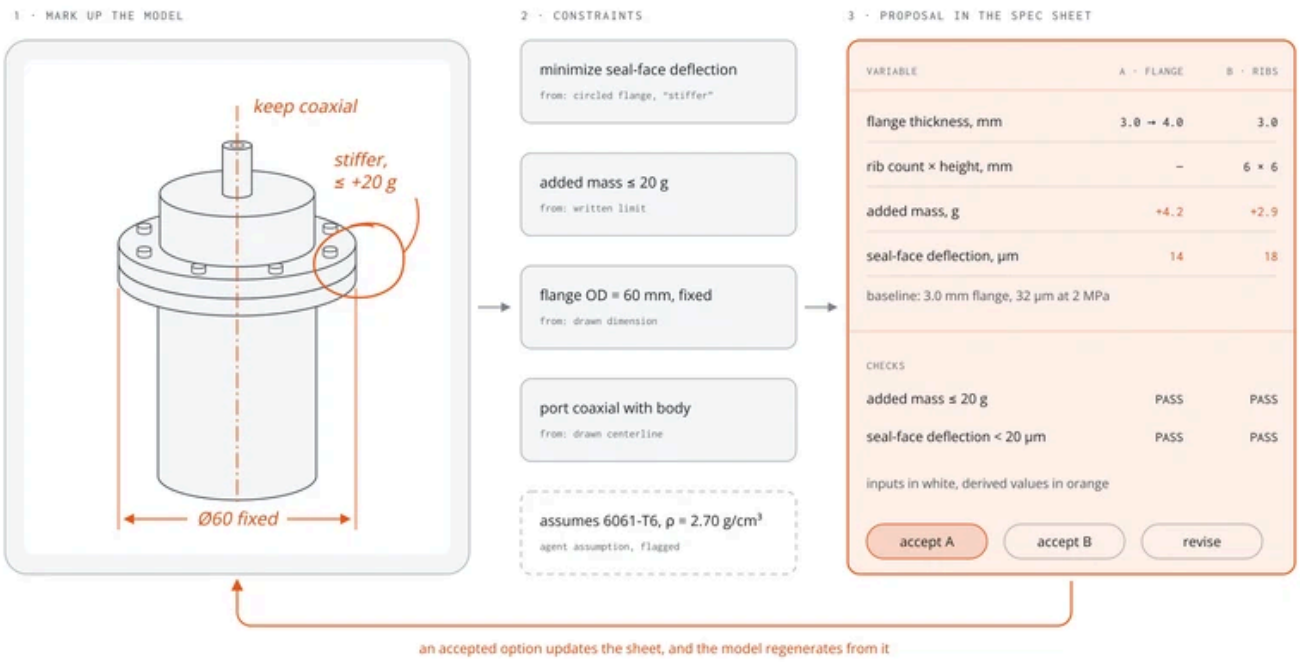


FIG. 05 A sketch of an intent-driven CAD edit. Marks on the tablet become explicit constraints, and the agent answers with options written as rows in the specification sheet, each showing its consequences before I accept it. The part and the numbers are illustrative.

What matters is what happens behind the drawing. Each mark should become an explicit, traceable constraint, rather than a nudge to the geometry. The agent should keep a parametric record of intent, separate my requirements from its own assumptions, and return proposed edits with their consequences for me to accept or reject. It should never quietly change the design basis. A beautiful model that can't be reliably modified, analyzed or traced back to its requirements isn't worth much.

The specification sheet is part of how I'm preparing for that. If every dimension already lives in one traceable place, the sheet is the obvious place for an agent's proposals to land, and handing geometry to an agent becomes a small step. Once a part exists in CAD, simulation becomes a side loop of its own: where the analytical models are too coarse, I run FEA on the geometry and iterate between the two until the part holds up, before anything is built.

11

Outlook

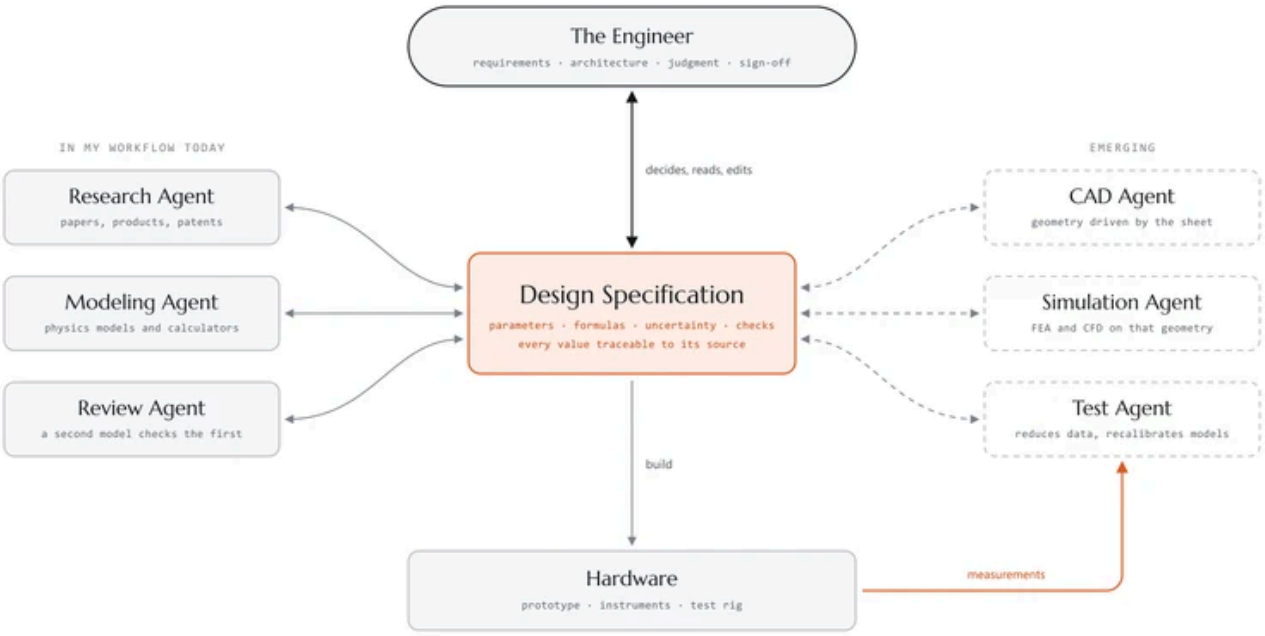


FIG. 06 A speculative workflow for mechanical engineering. Solid boxes are part of how I work now. Dashed boxes are where I expect agents to arrive.

For most of history, the ability to reason through hard technical problems has been one of the main limits on technology. Universities, professional societies and research labs exist partly to accumulate and pass on that scarce expertise. AI could relax that limit a great deal. Making something physically real still takes energy, material, time, factories, instruments and contact with a world we understand only in part. If intellectual work keeps getting cheaper, the gap between what we know how to build and what we have actually built will grow, and closing that gap looks to me like the biggest engineering opportunity of the next few decades.

Someone still has to decide what is worth building, choose the architecture and sign their name to the result, and traceability matters more in that future. The more work I delegate, the more I need to audit it quickly, which is why I keep the plain

spreadsheet and why I don't hand over work I can't check. The essay half of this piece is a hypothesis about where engineering is going. The workflow half is something I can run and improve now, and I expect to learn more about the hypothesis from running the workflow than from thinking about it.

12

Lessons

- Keep one source of truth, in a form a person can read.
- Prefer tools that show their work. A spreadsheet with its formulas printed, or a notebook, beats a polished interface.
- Treat every number as a measurement, with a source and an uncertainty, whether a person or an agent produced it.
- Treat one model's answer as a draft, and when two models disagree, treat the disagreement as the useful part.
- Passing checks is not validation. Only a test on hardware is.
- Invest in verifiers. The rig, the instrument and the error budget are what make an agent's speed usable.
- When designs are cheap, spend the effort on requirements that can rank them and tests that can eliminate them.
- Keep the decisions. Requirements, architecture and the final sanity check are mine.
- Save what generalizes, so the next project starts further along.

References

- D. Patel, "The avalanche that hit math is coming for AI development" (November 2026), link
- Navier–Stokes: HPCwire, "Unsolved for 90 years, OpenAI says AI cracked Millennium Prize problem in 88 hours" (September 2026), link
- YBCO: M. K. Wu et al., "Superconductivity at 93 K in a new mixed-phase Y-Ba-Cu-O compound system at ambient pressure," *Phys. Rev. Lett.* 58, 908 (1987). doi:10.1103/PhysRevLett.58.908
- REBCO fusion magnet: MIT News, "MIT-designed project achieves major advance toward fusion energy" (September 2021), link; and "Tests show high-temperature superconducting magnets are ready for fusion" (March 2024), link
- NdFeB magnets: Science Japan, "Honda Prize 2023 awarded to inventors of the neodymium magnet," link
- Lithium-ion batteries: Nobel Prize in Chemistry 2019, popular information, link
- Perovskite photovoltaics: PV Tech, "The hope and hype of commercial perovskites," link; Oxford PV, first commercial tandem module shipment (September 2024), link
- A-Lab: N. J. Szymanski et al., "An autonomous laboratory for the accelerated synthesis of inorganic materials," *Nature* 624, 86 (2023); the title originally read "novel materials." doi:10.1038/s41586-023-06734-w
- GNoME critique: A. K. Cheetham and R. Seshadri, "Artificial intelligence driving materials discovery? Perspective on the article: Scaling deep learning for materials discovery," *Chem. Mater.* 36, 3490 (2024). doi:10.1021/acs.chemmater.4c00643

- A-Lab critique: J. Leeman et al., "Challenges in high-throughput inorganic materials prediction and autonomous synthesis," *PRX Energy* 3, 011002 (2024). doi:10.1103/PRXEnergy.3.011002
- A-Lab correction: Helix (Northwestern Medill), "AI-generated crystals: scientists weigh risks and rewards of automated materials discovery" (April 2026), link
- FRDPARRC: A. Slocum, MIT 2.007 recitation notes, link
- Donaldson reversal: R. Grejda, E. Marsh and R. Vallance, "Techniques for calibrating spindles with nanometer error motion," *Precision Engineering* 29, 113 (2005). doi:10.1016/j.precisioneng.2004.05.003